

1 gain_loop 校准方案

1.1 gain_loop 校准环路结构

在整个校准系统中, gain_loop 校准是第二步。TIADC 的增益失配误差主要来源于各通道 ADC 的参考电压失配、器件失配和积分非线性失真等, 因此与失调失配误差一样, 每个通道的增益失配误差在一定时间内可以看成稳定不变的。gain_loop 校准是通过消除通道间的相对增益误差, 减小增益失配误差对采样数据的影响。gain_loop 校准的环路结构和数据位宽如下图 1 所示 (环路上的参数是对应 verilog 代码中的参数名, 数字为该参数的位宽):

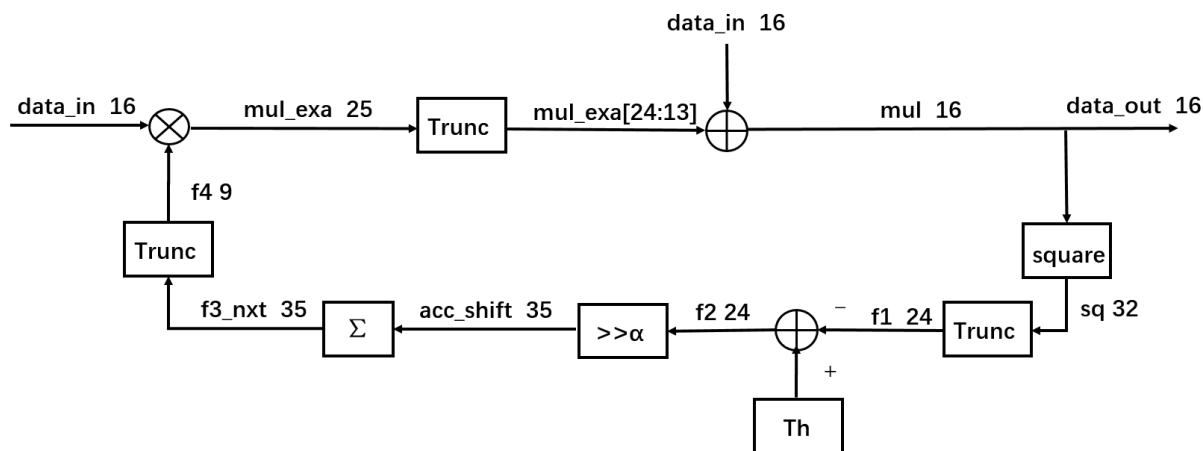


图 1: gain_loop 校准的环路结构和数据位宽

1.2 gain_loop 校准工作原理

由于工艺误差导致的通道间失配误差会影响 TIADC 的整体动态性能, 为了校准失配误差, 有两种方式: 一种是把每个通道的失配误差都消除, 从而实现通道间误差失配的校准; 另外一种消除通道间的相对失配误差, 即是基于参考通道的校准, 这里参考通道, 可以是 TIADC 以外增加的通道, 也可以是 TIADC 的任一自有通道。相比第一种方式, 第二种方式消除的是通道间的相对误差, 实现起来更容易实现, 而且校准效果也不会降低太多。

offset 校准中, 采用的是方式一, 通过多次迭代求出各个通道失调失配误差的近似值, 从而消除采样数据中的失调失配误差。但对于 gain_loop, 消除通道间的相对增益失配即可进行校准, 因为通道间增益误差的影响可以被看作幅度 (AM) 调制过程, 即输入采样信号被周期性的周期增益误差序列幅度调制, 因此, 不需要消除各个通道采样数据的增益误差, 只需要将各个通道的增益失配误差调整到同一个值 (即消除相对增益失配), 就能完成校准。校准完后相当于数据在 ADC 采样中幅值被整体放大或者缩小。

为了实现增益校准, 引入两个参数, 一个是参考值, 即阈值 (Th), 另一个是增益补偿系数。

假设 16 路通道的增益失配误差分别为 $g_1, g_2, \dots, g_{15}, g_{16}$, 使得各个通道的采样值由 $m[n]$ (理想采样值) 变成了 $(1 + g_i)m[n]$ (实际采样值), 相当于采样数据在各个通道得到不同程度的放大或者缩小。为了消除相对增益误差, 对各个通道的增益进行调整, 即引入增益补偿系数 $g'_1, g'_2, \dots, g'_{15}, g'_{16}$, 使采样数据变成 $(1 + g_i + g'_i)m[n]$, 并且满足式子:

$$1 + g_1 + g'_1 = 1 + g_2 + g'_2 = \dots = 1 + g_{15} + g'_{15} = 1 + g_{16} + g'_{16} \quad (1)$$

这样采样数据就相当于被整体放大或者缩小, 从而降低了增益失配误差带来的杂散。增益补偿系数 $g'_i[n]$ (i 是指第 i 通道, 取值范围为 $0, 1, \dots, 16$) 在环路中的作用是调整环路的输出, 用公式表示为:

$$\begin{aligned} y_i[n] &= x_i[n] + g'_i[n]x_i[n] \\ &= (1 + g'_i[n])x_i[n] \end{aligned} \quad (2)$$

那么, 如何调整每一路通道的增益补偿系数 g'_i , 使得公式 (1) 成立? 这就需要另一个参数, 即参考值 (阈值 Th)

该增益校准环路的工作原理就是: 不断增大或减小增益补偿系数 $g'_i[n]$, 使得输出数据 $y[n]$ 的平方 $y[n]^2$, 在一个周期或者数个周期 (亦或是很多次迭代次数) 的平均值等于 Th 的平均值, 即调整增益补偿系数, 使得每个通道输出信号的平均功率都等于 Th 的平均值, 从而消除各个通道的相对增益误差。这一过程是通过 LMS 迭代算法实现的, 当 $|y_i[n]^2|$ 大于 Th 时, 减小增益补偿系数, 当 $|y_i[n]^2|$ 小于 Th 时, 增大增益补偿系数, 即负反馈。用公式表示为:

$$g'_i[n+1] = g'_i[n] + \alpha(Th[n] - y_i[n]^2) \quad (3)$$

其中 i 表示为第 I 通道, n 表示为迭代数。 α 为带宽, 其取值范围为 $2^{-32}, 2^{-33}, 2^{-34}, 2^{-35}, 2^{-36}, 2^{-37}, 2^{-38}, 2^{-39}$ 。

经过大量数据的迭代, 各个通道的增益补偿系数会趋于收敛, 这时我们可以将其看成一个常数 (但并不是完全不变, 因为还在不断迭代, 因此会有细微纹波)。当增益补偿系数收敛后, 公式 (1) 就成立。

为了便于理解为什么增益补偿系数敛, 输出信号的平均功率即为 Th 的平均值且公式 (1) 成立, 我们做一个简单分析。当经过 N 个数据的迭代, 增益补偿系数 g' 和参考值 Th 都已经收敛, 我们可以认为这两个数不再变化, 即 $g'_i[N] = g'_i[N+1] = \dots = g'_i[N+n-1]$, 可以推导出:

$$g'_i[N+n-1] - g'_i[N] = 0 \quad (4)$$

利用公式 (3) 对 $g'_i[N+n-1]$ 进行展开, 可得

$$g'_i[N] + \alpha(Th[N] - y_i[N]^2) + \dots + \alpha(Th[N+n-1] - y_i[N+n-1]^2) - g'_i[N] = 0 \quad (5)$$

设参考值 Th 的平均值为 $\overline{Th}$ ，因此可以认为 $Th[N] + Th[N+1] + \dots + Th[N+n-1] = n\overline{Th}$ ，化简后可得：

$$\frac{y_i[N]^2 + y_i[N+1]^2 + y_i[N+2]^2 + \dots + y_i[N+n-1]^2}{n} = \overline{Th} \quad (6)$$

由公式 (6) 可得，当增益补偿系数收敛后，输出信号的平均功率等于 $\overline{Th}$ ，即 16 路通道增益补偿后的输出信号的平均功率都相等且等于 $\overline{Th}$ 。

因此，即便 16 路子 ADC 的增益失配误差不同，但使用同一参考信号对其迭代进行约束，即可在增益补偿系数收敛后，使得公式 (1) 成立。各个通道的相对增益误差就被消除了，数据相当于整体被放大或者缩小。

1.3 工作模式

gain_loop 校准模块包含两种工作状态：数据校准中状态和数据校准完成状态。

在数据校准中状态时，gain_loop 校准模块从 offset 校准模块通道中得到的每一个数据，需要进行如图 1 所示的环路的计算，模块对增益补偿系数的初始值设为 0，经过大量数据的迭代（该过程中 α 在不断调小），增益补偿系数 g'_i 收敛，此时 gain_loop 校准完成。在未收敛之前，输入数据的 $(1 + g'_i)$ 倍（参数 mul）不能输入到下一模块（time skew 校准模块），因为增益补偿系数还未收敛，会影响 time skew 校准模块的精度。

在数据校准完成状态，只需要将输入数据的 $(1 + g'_i)$ 倍（参数 mul）传递到 time skew 校准模块即可，而不需要继续在回路中迭代。

图 2 展示了 $\alpha = 2^{-39}$ 时，数据在环路中迭代的位宽变换，其中变量名对应 verilog 中变量名称。长竖线为小数点位置，带有数字的方框是包含实际数据的位宽，空白框为扩展的位宽，为了使位宽扩展对加减法运算结果不产生影响，需要高位补符号位，低位补 0。

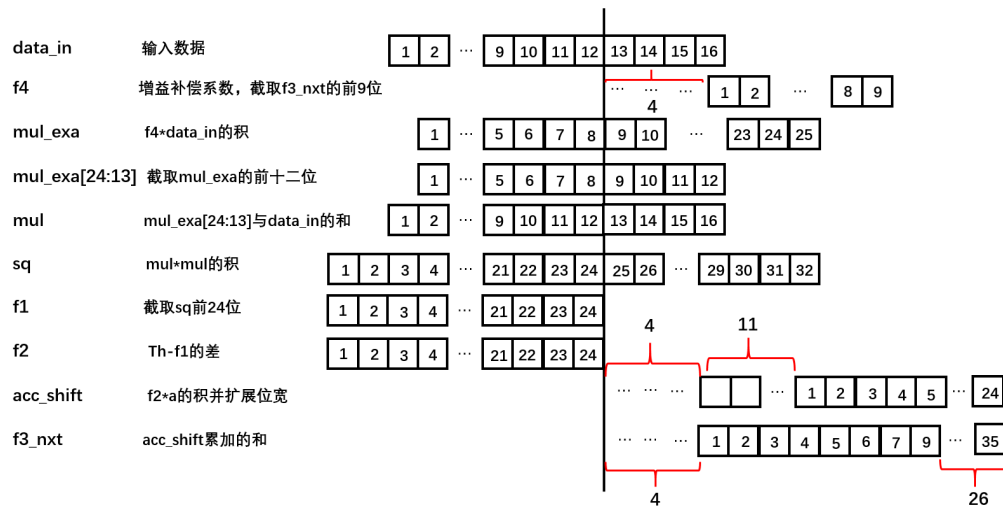


图 2: gain_loop 校准的环路结构和数据位宽

1.4 模块结构

1.4.1 溢出保护模块

图三为各个通道增益补偿系数不断迭代收敛的曲线，可以看到不同通道的 g_i 初始值均为 0，在不断的迭代计算中，逐渐收敛。从整体图来看，各个通道的增益补偿系数收敛成了一条直线。但挑选其中一条直线放大之后，由图四局部图可以观察到，其实增益补偿系数会上下抖动。实验可得，带宽（ α ）越大，抖动幅度越大。

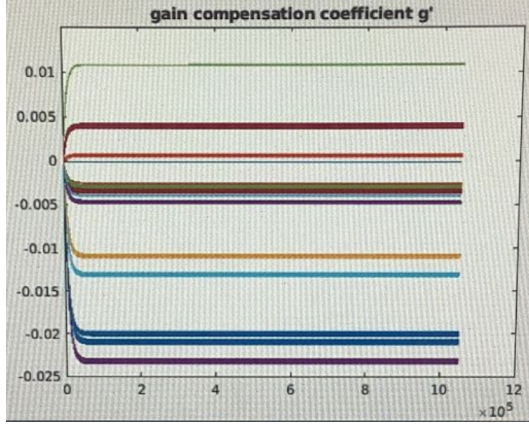


图 3: g'_i 增益补偿系数迭代整体图

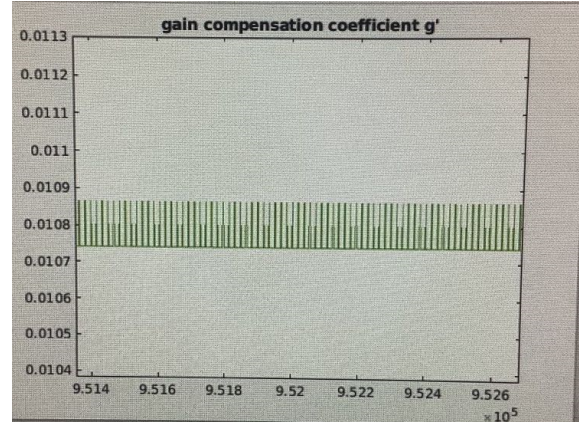


图 4: g'_1 增益补偿系数迭代局部图

基于通道的增益失配误差的范围，增益补偿系数的范围设定为-3%~+3%，精度设定为 0.05%。由图 2 可以看出 f4(增益补偿系数在 verilog 代码中的参数名) 的最高位（符号位）为小数点后第五位，且其位宽为 9。因此，其范围为 $\frac{-2^8}{2^{13}} \sim \frac{2^8-1}{2^{13}}$ ，即-0.03125~0.0311279，满足范围设定。又因为其最低位在小数点后 13 位，即 $2^{-13}=0.000122$ ，说明实际精度已经接近 0.01%。

由以上分析可知，受到硬件资源的限制，存储增益补偿系数的参数也有上限和下限，因此，就像 offset 校准一样，我们要考虑进行溢出保护，因为原理和 offset 校准溢出保护相似，所以我们不再赘述，只是总结一下，当发生上溢时，将 011_1111_1111_1111_1111_1111_1111_1111 存入 f3_nxt，当发生下溢时，将 100_0000_0000_0000_0000_0000_0000_0000 存入 f3_nxt。

1.4.2 档位选择模块

图 5 和图 6 展示了不同的 α 值对增益补偿系数迭代的影响。显然， α 越小，offset 估计值就需要更多的迭代次数、迭代数据来到达收敛。这会造成时间上的浪费，但这并不意味着 α 越大就越好。把整体图放大，观察其中一个通道的增益补偿系数在收敛时的情况，就能发现新的不同。通过图 7 和图 8 的对比，可以看出，当 $\alpha = 2^{-35}$ 时，增益补偿系数收敛时的抖动幅度大约在 0.00015，而当 $\alpha = 2^{-39}$ 时，增益补偿系数在放大很多倍时依然为一条直线。也就是说，当 α 越小，增益补偿系数收敛时的抖动幅度越小。

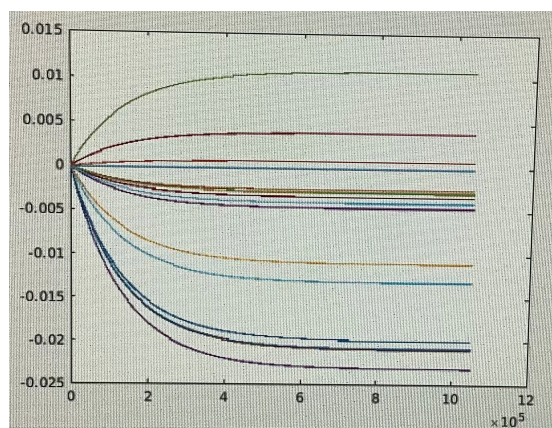
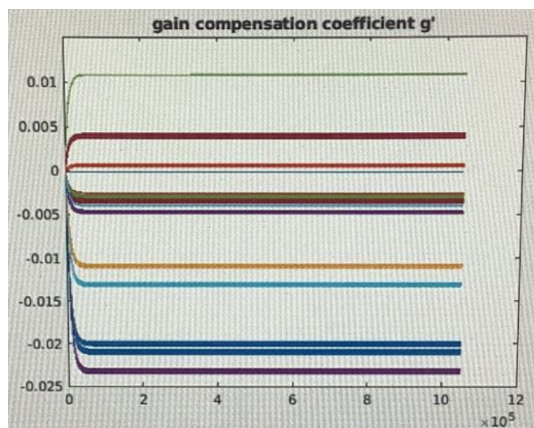


图 5: 增益补偿系数 g' 迭代 ($\alpha = 2^{-35}$) 整体图 图 6: 增益补偿系数 g' 迭代 ($\alpha = 2^{-39}$) 整体图

因此，为了同时保证收敛的速度和精度，将 α 设置八个档位，分别为 2^{-32} , 2^{-33} , 2^{-34} , 2^{-35} , 2^{-36} , 2^{-37} , 2^{-38} , 2^{-39} 。初始档位为 $\alpha = 2^{-32}$ ，使得增益补偿系数快速到达收敛。当收敛判断模块认为当前档位的增益补偿系数已经收敛，则会调到下一档位，即 $\alpha = 2^{-33}$ 。以此类推，直到档位调到 $\alpha = 2^{-39}$ 时，且收敛判断模块认为当前档位增益补偿系数已经收敛，则 gain_loop 校准中状态结束，开始 gain_loop 校准完成状态。

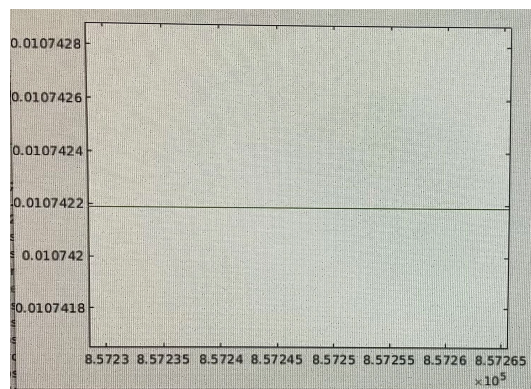
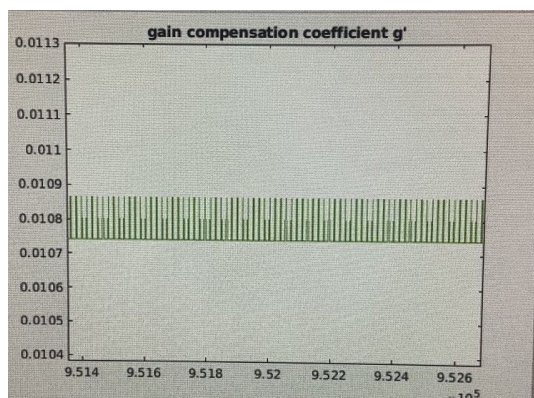


图 7: 增益补偿系数 g' 迭代 ($\alpha = 2^{-35}$) 局部图 图 8: 增益补偿系数 g' 迭代 ($\alpha = 2^{-39}$) 局部图

1.4.3 参考值选择模块

参考值的选取有两种模式，第一种是可以选择 16 路通道的任一通道为参考通道，并以该通道的功率作为参考值来校准其他通道，通常选择第一通道作为参考通道。另一种是计算十六个通道的输入数据的功率，然后求其平均值，作为参考值。选取其中一种模式作为参考值，在参考值进入环路前，要先通过 leaky 积分器，其实质相当于一个低通滤波器，作用是降低参考值波形的抖动。参考值选取模块其在环路中的结构如图 9 所示：

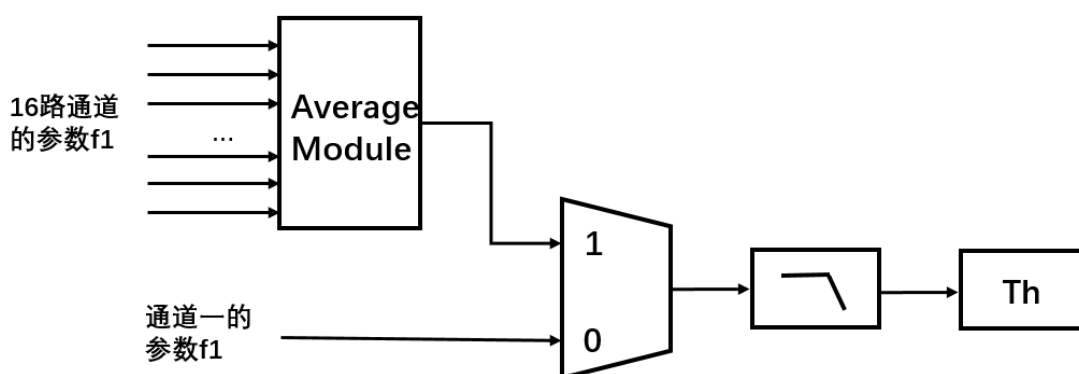


图 9: 参考值选取模块结构

图 10 是采用模式一时，参考值的波形图，看起来像一个正方形，其实是因为一直在上下波动，只不过比较密集，因此像一个正方形，这个形状也符合我们的预期，因为一个通道的输入数据是余弦信号，其平方也应该在 0 和最大值之间来回波动。值得注意的是，增益校准，其实是靠参考通道的平均功率来进行校准的，因此虽然模块一参考通道瞬时功率一直在变化，但只要其平均功率不变，就会让其他十五个通道的平均功率向着参考通道的平均功率收敛，进而消除相对增益误差。

图 11 是采用模式二时，参考值的波形图，因为选取十六个通道 $(1+g'_i)$ 倍输入数据的瞬时功率的平均值作为参考值，所以在迭代过程中参考值基本保持不变，波动范围很小。

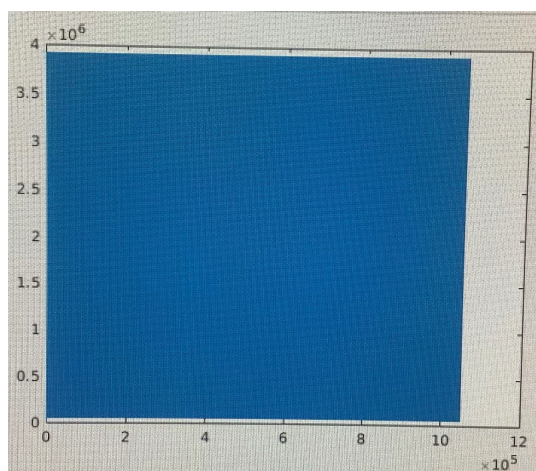


图 10: 选取模块一时参考值迭代过程波形图

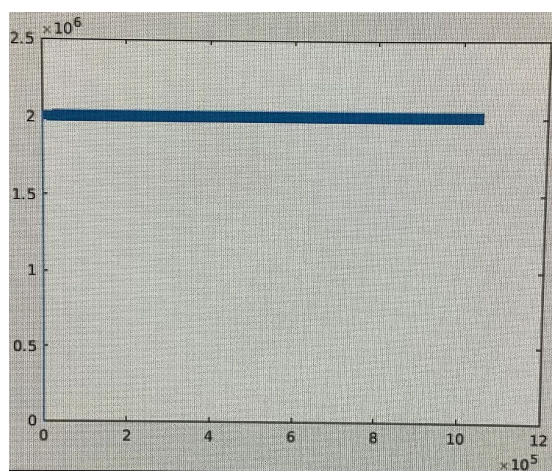


图 11: 选取模块二时参考值迭代过程波形图

1.4.4 收敛判断模块

1.5 verilog 代码参数

表 1: gain_loop 校准参数及功能

分类	参数名	功能
输入参数	rst_n	清零信号。
	clk	时钟信号。
	mode	控制参考值选取模式 0 当 mode 为 0 时，选择一通道输入数据的瞬时功率作为参考值，当 mode 为 1 时，选择十六个通道 $(1 + g'_i)$ 倍输入数据的瞬时功率的平均值作为参考值。
	bw_gear	3 位宽控制字，控制 8 种校准步长（带宽），8 个档位对应的带宽分别为 2^{-32} , 2^{-33} , 2^{-34} , 2^{-35} , 2^{-36} , 2^{-37} , 2^{-38} , 2^{-39} 。在迭代过程中，控制字控制带宽不断变小，使得增益补偿系数收敛速度较快的同时，抖动尽可能小。如果控制字出现 default 的情况，则令带宽设置为 2^{-37} 。
	data_in	16 位宽，有符号数。offset 校准完成后，将数据输入到 gain_loop 校准环路中。即需要校准的数据。
	avg	24 位宽，有符号数。需要注意的是，通道一调用的是 ref_gain_cal.v，剩下的十五个通道调用的是 gain_cal.v。当 mode 为 0 时，avg 对 ref_gain_cal.v 来说是输出参数，即将通道一输入数据求瞬时功率后，输出给剩下的十五个通道，avg 对于 gain_cal.v 是输入参数。通道一的输入数据瞬时功率 avg 输出后，通过端口 avg 输入到剩下的十五个通道，同时通过端口 mean 输入到通道一，作为十六个通道的参考值。当 mode 为 1 时，avg16 作为参考值。
	avg16	24 位宽，有符号数。当 mode 为 1 时，在 average.v 中得到十六个通道输入数据瞬时功率的平均值，再经过 leaky 积分器后，作为 avg16 输入到 16 个通道中作为参考值。
	gain_step	1bit，为 1 的时候说明 gain_loop 校准还未完成，为 0 的时候说明 gain_loop 校准已经完成。
	mean	24 位宽，有符号数。为 ref_gain_cal.v 的输入参数，是通道一输入数据的瞬时功率，在 mode=0 时，作为通道一校准的参考值。

输出参数	avg	24 位宽，有符号数。avg 为 ref_gain_cal.v 的输出参数，是通道一输入数据 $\ast(1+g')$ 的瞬时功率（平方），经过 leaky 积分器滤波后从模块输出。
	data_out	当增益补偿系数 g' 收敛后， $data_out=(1+g')data_in$ ，消除了通道间的相对增益误差。增益补偿系数位收敛时，不会向下一模块输出数据。
	f1	24 位宽，有符号数。sq 是 mul 数据的平方，32 位宽，f1 为截取 sq[31:8] 的值，在 mode=1 时，16 路通道都把自己的 f1 输出到 average.v 中求平均。
内部参数	sq	32 位宽，有符号数。是 mul 的平方。
	f2	24 位宽，有符号数。f2 为 Th-f1 的值。
	f3_nxt	35 位宽，有符号数。存储 acc 的累加值，当累加出现溢出时，则根据是上溢还是下溢，分别存入最大值或者最小值。
	f3	35 位宽，有符号数。f3=f3_nxt。
	f4	9 位宽，有符号数。f4 为截取 f3_nxt 前 9 位的值，作为增益补偿系数，与输入数据相乘。
	mul_exa	25 位宽，有符号数。等于 $f4 \ast data_in$ 。为输入数据和增益补偿系数的乘积。
	mul	16 位宽，有符号数。等于 $data_in + mul_exa[24:13]$ 。等同于 $(1+g')data_in$ 。
	acc_shift	35 位宽，有符号数。存储的是 $\alpha \ast f2$ 的值，相当于将 f2 向右移位。高位补 f2 的符号位，低位补 0。
	acc	35 位宽，有符号数。等于 $f3 + acc_shift$ 。其符号位是判断是否有溢出的重要依据
	th	24 位宽，有符号数。在增益校准中作为参考值，根据 mode 的值不同，选取第一通道输出的 avg 或者选取 average.v 计算的 16 通道瞬时功率均值 avg16 作为参考值。
	lp	24 位宽，有符号数。等于 25 位宽的 lp_nxt[n] 四舍五入的值，因此减小了一位。
	lp_nxt	寄存器变量，25 位宽，有符号数。当时钟上升沿时， $lp_nxt[n]=0.5lp[n-1]+0.5 \ast f1$ ，leaky 积分器，相当于低通滤波。
	overflow_pos	检测 f3_nxt 累加过程中是否有上溢产生，等于 $\sim f3[34] \& \sim f2[23] \& acc[34]$ ，若有上溢，则将 011_1111_1111_1111_1111_1111_1111_1111_1111_1111 存入 f3_nxt，若无，则将 acc 存入。
	overflow_neg	检测 f3_nxt 累加过程中是否有上溢产生，等于 $f3[34] \& f2[23] \& \sim acc[34]$ ，若有上溢，则将 100_0000_0000_0000_0000_0000_0000_0000_0000_0000 存入 f3_nxt，若无，则将 acc 存入。